

Introducción a React (2ª parte)

Uso de props

Actualmente, el componente **Alert** es bastante inflexible. Por ejemplo, el usuario de la alerta no puede cambiar el título ni el mensaje. Por el momento, el título o el mensaje deben cambiarse dentro del propio **Alert**. Los **props** resuelven este problema, y los veremos en esta sección.

NOTA

«Props» es la abreviatura de «properties» (propiedades). La comunidad de React suele referirse a ellas como «props», por lo que lo haremos también en este tema.

Comprender los props

El parámetro **props** es un parámetro opcional que se pasa a un componente de React. Este parámetro es un objeto que contiene las propiedades que elijamos, lo que permite a un componente padre pasar datos. El siguiente fragmento de código muestra un parámetro **props** en un componente **ContactDetails**:

```
function ContactDetails(props) {  
  console.log(props.name);  
  console.log(props.email);  
  ...  
}
```

El parámetro **props** contiene las propiedades **name** y **email** en el fragmento de código anterior.

NOTA

*El parámetro no tiene por qué llamarse **props**, pero es una práctica habitual.*

Las **props** se pasan a un componente en JSX como atributos. Los nombres de las props deben coincidir con lo definido en el componente. A continuación, se muestra un ejemplo de cómo pasar props al componente **ContactDetails** anterior:

```
<ContactDetails name="Fred" email="fred@somewhere.com" />
```

Por lo tanto, los **props** hacen que la salida del componente sea flexible. Los usuarios del componente pueden pasar los **props** adecuados al componente para obtener la salida deseada.

A continuación, añadiremos algunos **props** al componente **Alert** en el que hemos estado trabajando.

Añadir props al componente Alert

En el proyecto, sigue estos pasos para añadir **props** al componente **Alert** y hacerlo más flexible:

Empieza por ejecutar la aplicación en modo de desarrollo si aún no se está ejecutando. Para ello, ejecuta el comando **npm run dev** en la terminal.

Abre **Alert.jsx** y añade un parámetro **props** a la función:

```
export function Alert(props) {  
  ...  
}
```

Definiremos las siguientes propiedades para la alerta:

type: Puede ser «**information**» o «**warning**» y determinará el icono de la alerta.

heading: determinará el título de la alerta.

children: Determinará el contenido de la alerta. La propiedad **children** es, en realidad, una propiedad especial que se utiliza para el contenido principal de un componente.

Actualiza el código JSX del componente «**Alert**» para utilizar las propiedades de la siguiente manera:

```
export function Alert(props) {  
  return (  
    <div>  
      <div>  
        <span  
          role="img"  
          aria-label={  
            props.type === 'warning'  
              ? 'Advertencia'  
              : 'Información'  
            }  
        >  
          {props.type === 'warning' ? '⚠️' : 'ℹ️'}  
        </span>  
        <span>{props.heading}</span>  
      </div>  
      <div>{props.children}</div>  
    </div>  
  );  
}
```

Es posible que observes que el componente **Alert** del navegador ahora no muestra nada más que un icono de información (se trata de un emoji de información); esto se debe a que el componente **App** aún no está pasando ninguna propiedad a **Alert**.

Abre **App.jsx** y actualiza el componente **Alert** en el JSX para pasarle props de la siguiente manera:

```
export default function App() {  
  return (  
    <div className="App">  
      <Alert type="information" heading="Success">  
        ¡Todo va muy bien!  
      </Alert>  
    </div>  
  );  
}
```

```

        </Alert>
      </div>
    );
  }

```

Fíjate en que el componente **Alert** ya no se cierra automáticamente, por lo que se puede pasar «¡**Todo va muy bien!**» a su contenido. El contenido se pasa a la propiedad **children**.

La aplicación muestra ahora el componente **Alert** configurado:

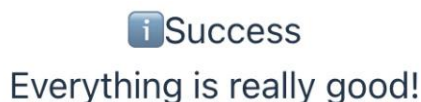


Figura 1.13 – Componente Alert configurado en la aplicación

Podemos limpiar un poco el código del componente **Alert** desestructurando el parámetro **props**.

NOTA

La desestructuración es una característica de JavaScript que permite extraer propiedades de un objeto. Para obtener más información, consulta el siguiente enlace:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring_assignment.

Vuelve a abrir el archivo **Alert.jsx**, desestructura el parámetro **de la función** y utiliza las propiedades desestructuradas de la siguiente manera:

```

export function Alert({ type, heading, children }) {
  return (
    <div>
      <div>
        <span
          role="img"
          aria-label={
            type === 'warning' ? 'Advertencia' : 'Información'
          }
        >
          {type === 'warning' ? '⚠️' : 'ℹ️'}
        </span>
        <span>{heading}</span>
      </div>
      <div>{children}</div>
    </div>
  );
}

```

Esto queda un poco más limpio porque usamos las propiedades desempaquetadas directamente en lugar de tener que hacer referencia a ellas a través del parámetro **props**.

Queremos que la propiedad **type** tenga como valor predeterminado «**information**».

Define este valor predeterminado de la siguiente manera:

```
export function Alert({
  type = 'information',
  heading,
  children
}) {
  ...
}
```

Con esto queda completada, por ahora, la implementación de los **props** en el componente **Alert**. A continuación, un breve resumen sobre los **props**:

- Los **props** permiten que un componente sea configurado por el JSX que lo utiliza y se pasan como atributos JSX
- Los **props** se reciben en la definición del componente en un parámetro de objeto y luego se pueden utilizar en su JSX

A continuación, continuaremos perfeccionando el componente **Alert** permitiéndole al usuario cerrarlo.

Uso del estado

El **estado** de un componente de React es una variable especial que puede cambiar a lo largo del ciclo de vida de un componente. En esta sección, aprenderemos sobre la variable **de estado** y la utilizaremos dentro de nuestro componente **Alert**. Usaremos **el estado** para permitir que el usuario cierre la alerta.

Comprender el estado

No existe una lista predefinida de estados; nosotros definimos lo que es apropiado para un componente determinado. Algunos componentes ni siquiera necesitarán ningún estado: el componente **Alert** no ha requerido un estado para los requisitos hasta ahora.

El estado es una parte clave para hacer que un componente sea interactivo. Cuando un usuario interactúa con un componente, es posible que la salida del componente tenga que cambiar. Un cambio en el estado hace que el componente se actualice, lo que se conoce más comúnmente como «**re-render**».

El estado se define utilizando la función **useState** de React. La función **useState** es uno de **los Hooks** de React.

La sintaxis de **useState** es la siguiente:

```
const [state, setState] = useState(initialState);
```

Estos son los puntos clave:

El valor del estado inicial se pasa a **useState**. Si no se pasa ningún valor, inicialmente será **indefinido**.

useState devuelve una tupla que contiene el valor del estado actual y una función para actualizar dicho valor. La tupla se desestructura en el fragmento de código anterior.

En el fragmento de código anterior, el nombre de la variable de estado es «**state**», pero podemos elegir cualquier nombre significativo.

También podemos elegir el nombre de la función de configuración del estado, pero es habitual utilizar el mismo nombre que la variable de estado precedido por «**set**».

Se pueden definir varios estados creando varias instancias de **useState**. Por ejemplo, estas son las definiciones para los estados de carga y de error:

```
const [loading, setLoading] = useState(true);  
const [error, setError] = useState();
```

A continuación, implementaremos **el estado** en el componente **Alert** para determinar si es visible o no.

Implementación de un estado visible en el componente **Alert**

Comenzaremos implementando una función en el componente **Alert** que permita al usuario cerrarlo. Una parte clave de esa función es controlar la visibilidad de la alerta, lo cual haremos con un estado **visible**. Este estado será **true** o **false** y, inicialmente, se establecerá en **true**.

Sigue estos pasos para implementar un estado **visible** en **Alert**:

Si la aplicación aún no se está ejecutando, hazlo ejecutando el comando **npm run dev** en la terminal.

Abre **Alert.jsx** en el proyecto.

Añade la siguiente instrucción **de importación** al principio del archivo para importar el hook **useState** de React:

```
import { useState } from 'react';
```

Define el estado **visible** de la siguiente manera en la definición del componente:

```
export function Alert(...) {  
  const [visible, setVisible] = useState(true);  
  return (  
    ...  
  );  
}
```

Después de la declaración del estado, añade una condición que devuelva **null** si el estado **visible** es **false**. Esto significa que no se renderizará nada:

```
export function Alert(...) {  
  const [visible, setVisible] = useState(true);  
  if (!visible) {  
    return null;  
  }  
  return (  
    ...  
  );  
}
```

El componente se mostrará en la aplicación igual que antes porque el estado **visible** es **true**. Intenta cambiar el valor del estado inicial a **false** y verás que desaparece.

Actualmente, el componente **Alert** utiliza el valor del estado **visible** para no mostrar nada si es **false**. Sin embargo, el componente aún no actualiza el estado **visible**; es decir, **setVisible** no se utiliza por el momento. Actualizaremos el estado **visible** después de implementar un botón **de cierre**, lo cual haremos a continuación.

Añadir un botón de cierre a Alert

Añadiremos un botón **de cierre** al componente **Alert** para que el usuario pueda cerrarlo. Lo haremos configurable para que el consumidor de la alerta pueda elegir si se muestra el botón **de cierre**.

Sigue estos pasos:

Empieza abriendo el archivo **Alert.jsx** y añadiendo una propiedad **«closable»**:

```
export function Alert({  
  type = "information",  
  heading,  
  children,  
  closable  
}) {  
  ...  
}
```

El usuario del componente **Alert** utilizará la propiedad **closable** para especificar si aparece el botón **de cierre**.

Añade un botón **de cierre** entre el encabezado y el contenido de la siguiente manera:

```

export function Alert(...) {
  ...
  return (
    <div>
      <div>
        ...
        <span>{título}</span>
      </div>
      <button aria-label="Cerrar">
        <span role="img" aria-label="Cerrar">✕ </span>
      </button>
      <div>{children}</div>
    </div>
  );
}

```

Observe que al elemento **span** que contiene el icono **de cierre** se le asigna un rol «**img**» y una etiqueta «**Cerrar**» para facilitar la lectura a los lectores de pantalla. Del mismo modo, al botón también se le asigna una etiqueta «**Cerrar**» para facilitar la lectura a los lectores de pantalla.

El botón **de cierre** aparece en el componente **Alert** de la siguiente manera:

 Success



Everything is really good!

Figura 1.14 – El botón de cierre en el componente Alert

Por el momento, el botón **de cierre** siempre se mostrará, en lugar de solo cuando la propiedad **closable** sea **true**. Podemos utilizar una expresión lógica **AND** de cortocircuito de JavaScript (representada por los caracteres **&&**) para mostrar el botón **de cierre** de forma condicional. Para ello, realiza los siguientes cambios resaltados:

```

import { useState } from 'react';
export function Alert(...) {
  ...
  return (
    <div>
      <div>
        ...
        <span>{título}</span>
      </div>
      {closable && (
        <button aria-label="Cerrar">
          <span role="img" aria-label="Cerrar">✕

```

```

        </span>
      </button>
    )}
  <div>{children}</div>
</div>
);
}

```

Si «**closable**» es un valor **falso**, la expresión se **interrumpirá prematuramente** y, por lo tanto, no se mostrará el botón. Sin embargo, si «**closable**» es un **valor verdadero**, el botón se mostrará.

NOTA

Consulta el siguiente enlace para obtener más información sobre las expresiones lógicas **AND** de cortocircuito: [Operador lógico AND \(&&\) - JavaScript | MDN](https://developer.mozilla.org/en-US/docs/Glossary/Falsey)

Consulta el siguiente enlace para conocer los valores falsos de JavaScript:

<https://developer.mozilla.org/en-US/docs/Glossary/Falsey> y [Truthy - Glosario de MDN Web Docs | MDN](https://developer.mozilla.org/en-US/docs/Glossary/Truthy) para los valores verdaderos.

Abre **App.jsx** y pasa la propiedad **closable** a **Alert**:

```

export default function App() {
  return (
    <div className="App">
      <Alert
        type="information"
        heading="Success"
        closable
      >
        ¡Todo va muy bien!
      </Alert>
    </div>
  );
}

```

Fíjate en que no se ha definido explícitamente ningún valor en el atributo **closable**.

Podríamos haber pasado el valor de la siguiente manera:

```
closable={true}
```

Sin embargo, no es necesario pasar el valor en un atributo booleano. Si el atributo booleano está presente en un elemento, su valor es automáticamente **true**.

Cuando se especifica el atributo **closable**, el botón **de cierre** aparece en el componente **Alert** como lo hacía antes, en la *Figura 1.13*. Cuando no se especifica el atributo **closable**, el botón **de cierre** no aparece:



Figura 1.15 – El botón de cierre no aparece en el componente Alert cuando no se especifica closable

Aquí tienes un breve resumen de lo que hemos aprendido hasta ahora sobre el estado de React:

El estado se define utilizando el hook **useState** de React

El valor inicial del estado se puede pasar al hook **useState**

useState devuelve una variable de estado que se puede utilizar para renderizar elementos de forma condicional

useState también devuelve una función que se puede utilizar para actualizar el valor del estado

Quizás hayas notado que el botón **de cerrar** no cierra realmente la alerta. En la siguiente sección, solucionaremos esto mientras aprendemos sobre los eventos en React.

Uso de eventos

Los eventos son otro elemento clave para que un componente sea interactivo. En esta sección, comprenderemos qué son los eventos de React y cómo utilizarlos en elementos DOM. También aprenderemos a crear nuestros propios eventos de React.

Seguiremos ampliando la funcionalidad del componente **Alert** a medida que aprendamos sobre los eventos. Empezaremos por terminar la implementación del botón **de cierre** antes de crear un evento para cuando se haya cerrado la alerta.

Comprender los eventos

Los eventos del navegador se producen cuando el usuario interactúa con los elementos DOM. Por ejemplo, al hacer clic en un botón se genera un evento **de clic** desde ese botón.

Se puede ejecutar una lógica cuando se genera un evento. Por ejemplo, una alerta se puede cerrar al hacer clic en su botón **de cierre**. Se puede registrar en un elemento una función llamada **controlador de eventos** (a veces denominada «**escucha de eventos**») para un evento, que contiene la lógica que se ejecutará cuando ocurra ese evento en particular.

NOTA

Consulta el siguiente enlace para obtener más información sobre los eventos del navegador: https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Building_blocks/Events.

Los eventos en React son muy similares a los eventos nativos del navegador. De hecho, los eventos de React son una capa que envuelve los eventos nativos del navegador.

Los controladores de eventos en React suelen registrarse en un elemento en JSX mediante un atributo. El siguiente fragmento de código registra un controlador de eventos **de clic** llamado **handleClick** en un elemento **de botón**:

```
<button onClick={handleClick}>...</button>
```

A continuación, volveremos a nuestro componente **Alert** e implementaremos un controlador **de clic** en el botón **de cierre** que cierra la alerta.

Implementación de un controlador de clic del botón de cierre en la alerta

Por el momento, nuestro componente **Alert** contiene un botón **de cierre**, pero no ocurre nada al hacer clic en él. La alerta también contiene un estado **visible** que determina si se muestra la alerta. Por lo tanto, para completar la implementación del botón **de cierre**, necesitamos añadir un controlador de eventos que, al hacer clic en él, establezca el estado **visible** en **false**. Sigue estos pasos para hacerlo:

Si la aplicación aún no se está ejecutando, hazlo ejecutando el comando **npm run dev** en la terminal.

Abre **Alert.jsx** y registra un controlador **de clic** en el botón **de cierre** de la siguiente manera:

```
<button aria-label="Close" onClick={handleCloseClick}>
```

Hemos registrado un controlador **de clic** llamado **handleCloseClick** en el botón **de cierre**.

A continuación, debemos implementar la función **handleCloseClick** en el componente. Crea una función vacía para empezar, justo encima de la instrucción **return**:

```
export function Alert(...) {
  const [visible, setVisible] = useState(true);
  if (!visible) {
    return null;
  }
  function handleCloseClick() {}
  return (
    ...
  );
}
```

Esto puede parecer un poco extraño porque hemos colocado la función **handleCloseClick** dentro de otra función, **Alert**. El controlador debe estar dentro de la función **Alert** para que tenga acceso a **props** y **state**.

Si se prefiere, se puede utilizar la sintaxis de funciones flecha para los manejadores de eventos. Una versión del manejador como función flecha es la siguiente:

```
export function Alert(...) {
  const [visible, setVisible] = useState(true);
  if (!visible) {
    return null;
  }
  const handleCloseClick = () => {}
  return (
    ...
  );
}
```

Los controladores de eventos también se pueden añadir directamente al elemento en JSX de la siguiente manera:

```
<button aria-label="Cerrar" onClick={() => {}}>
```

En el componente **Alert**, nos ceñiremos a la función de controlador de eventos denominada **handleCloseClick**.

Ahora, podemos utilizar la función de configuración del estado **visible** para establecer dicho estado en «**false**» en el controlador de eventos:

```
function handleCloseClick() {
  setVisible(false);
}
```

Si haces clic en el botón **de cierre** de la aplicación, la alerta desaparece. ¡Genial!

Ten en cuenta que se puede utilizar la opción *de recarga* del navegador para recargar la aplicación y hacer que el componente **Alert** vuelva a aparecer.

A continuación, ampliaremos el botón **de cierre** para que genere un evento cuando se cierre la alerta.

Implementación de un evento de cierre de alerta

Ahora crearemos un evento personalizado en el componente **Alert**. El evento se activará cuando se cierre la alerta, de modo que los consumidores puedan ejecutar la lógica cuando esto ocurra.

Un evento personalizado en un componente se implementa utilizando una **prop**. La **prop** es una función que se invoca para generar el evento.

Para implementar un evento de cierre de alerta, sigue estos pasos:

Empieza abriendo **Alert.jsx** y añade una **prop** para el evento:

```
export function Alert({
  type = "information",
  heading,
  children,
  closable,
  onClose
}) {}
```

Hemos llamado a la propiedad **onClose**.

NOTA

Es habitual comenzar el nombre de una propiedad de evento con on.

En el controlador de eventos **handleCloseClick**, activa el evento de cierre después de establecer el estado **visible** en **false**:

```
function handleCloseClick() {  
  setVisible(false);  
  if (onClose) {  
    onClose();  
  }  
}
```

Fíjate en que solo invocamos **onClose** si está definido y el consumidor lo pasa como una propiedad. Esto significa que no estamos obligando al consumidor a gestionar este evento.

Ahora podemos gestionar el cierre de una alerta en el componente **App**. Abre **App.jsx** y añade el siguiente controlador de eventos a **Alert** en el JSX:

```
<Alert  
  type="information"  
  heading="Success"  
  closable  
  onClose={() => console.log("closed")}  
>  
  ¡Todo va muy bien!  
</Alert>;
```

En esta ocasión hemos utilizado un controlador de eventos en línea.

En la aplicación, si haces clic en el botón *de cerrar* y miras la consola, verás que se ha mostrado **«closed»**:



Figura 1.16 – Salida de la consola tras cerrar la alerta

Con esto concluimos el evento «close» y la implementación de la alerta para este tema.

Esto es lo que hemos aprendido sobre los eventos de React:

- Los eventos, junto con el estado, permiten que un componente sea interactivo
- Los controladores de eventos son funciones que se registran en elementos en JSX
- Se puede crear un evento personalizado implementando una propiedad de función e invocándola para generar el evento

- El componente que hemos creado en este tema es un componente de función. También puedes crear componentes utilizando clases. Sin embargo, los componentes de función son los predominantes en la comunidad de React. Estas son algunas de las razones:
- Por lo general, requieren menos código para su implementación
- La lógica dentro del componente se puede reutilizar más fácilmente
- Los React Hooks no se pueden utilizar en componentes de clase
 - Por estas razones, en este tema nos centraremos exclusivamente en los componentes de función.
- A continuación, aprenderemos a utilizar las herramientas de desarrollo de React para el navegador.

Uso de las herramientas de desarrollo de React

Las herramientas de desarrollo de React son una extensión de navegador disponible para Chrome, Firefox y Edge. Estas herramientas permiten inspeccionar y depurar aplicaciones React. En esta sección, vamos a instalar y utilizar estas herramientas en el componente **Alert** que hemos implementado en este tema.

Los enlaces a las extensiones son los siguientes:

Chrome: <https://chromewebstore.google.com/detail/react-developer-tools/fmkadmapgofadopljbjfkapdkoienihi>

Firefox: <https://addons.mozilla.org/en-GB/firefox/addon/react-devtools/>

Edge: <https://microsoftedge.microsoft.com/addons/detail/react-developer-tools/gpphkfbcpiddadnkolpfcpihlkkl>

Sigue las instrucciones del enlace correspondiente para instalar la extensión en tu navegador. Es posible que tengas que volver a abrir el navegador para que las herramientas estén disponibles.

Uso de la herramienta «Components»

La primera herramienta que vamos a explorar es la herramienta «**Components**». Te permite inspeccionar las propiedades y el estado actuales de un componente. Sigue estos pasos para probar esta herramienta en nuestro componente «**Alert**»:

Si la aplicación no se está ejecutando, iníciala ejecutando `npm run dev` en un terminal.

Abre las herramientas de desarrollo del navegador pulsando *F12* en Windows o *Cmd + Opción + I* en un Mac. Las herramientas de desarrollo de React se encuentran en dos paneles llamados «**Componentes**» y «**Perfilador**».

Selecciona el panel «**Componentes**». Aparecerá el árbol de componentes de React. Al seleccionar el componente «**Alert**» en el árbol de componentes, se muestra la información actual sobre él:

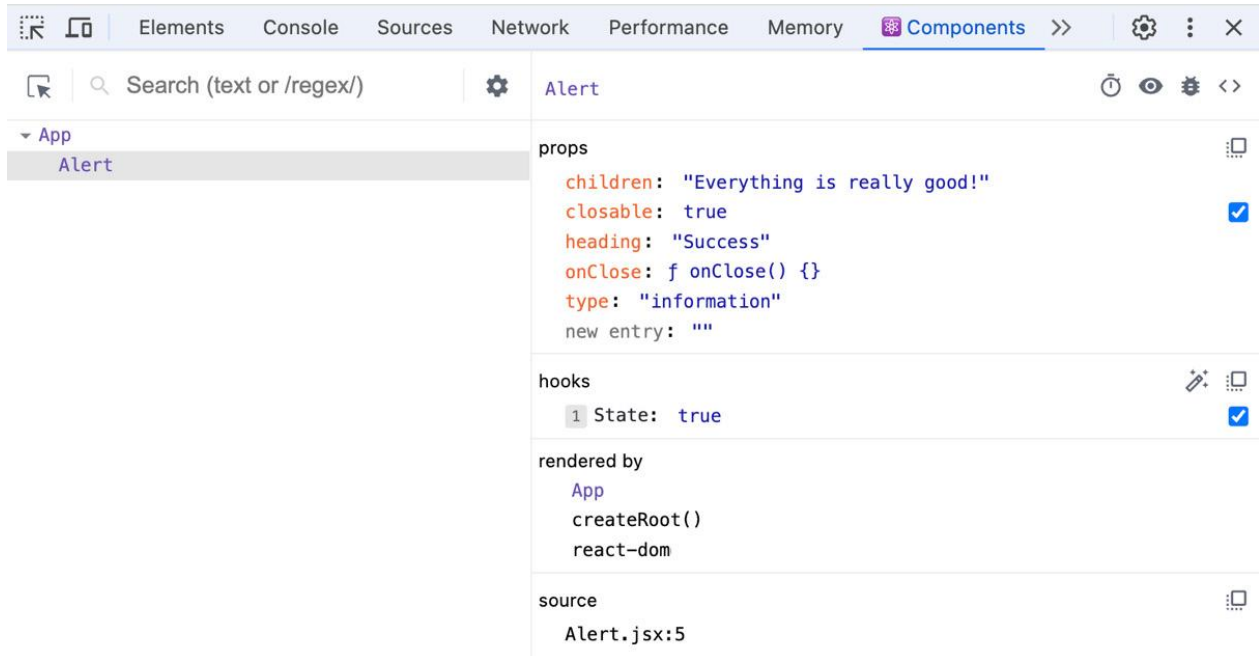


Figura 1.17 – Panel «Components» de las herramientas de desarrollo de React

Un conjunto útil de información sobre los componentes son los valores actuales de las propiedades. Puedes cambiar estos valores y la salida del componente en el navegador se actualizará. Prueba esto cambiando la propiedad **children**. Esta es una forma estupenda de comprobar manualmente que una propiedad funciona como se espera.

También hay una sección para los valores actuales de los Hooks, que incluye los valores de estado. Fíjate, sin embargo, en que el estado no tiene nombre: tiene un nombre genérico, **State**. No obstante, los Hooks aparecen en esta sección en el orden en que aparecen en el código del componente, por lo que podemos averiguar qué son estos elementos. Además, hay una opción «**Analizar nombres de Hooks**» (con un icono de asistente) que puede mostrar los nombres de las variables de los Hooks. Haz clic en esta opción para mostrar el nombre de la variable de estado entre paréntesis:

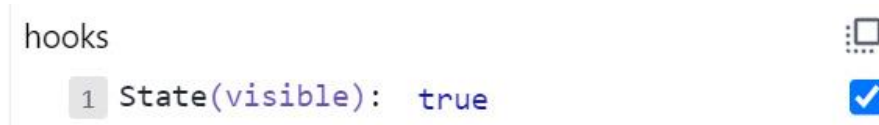


Figura 1.18 – Nombre de la variable de estado tras hacer clic en el icono del asistente

Vamos a observar el cambio en el valor del estado **visible**. Asegúrate de que **closable** esté activado en el componente (puedes establecer esta propiedad en **true** en la

sección **de propiedades** si no lo está). Al igual que con las propiedades, puedes cambiar los valores del estado utilizando las herramientas de desarrollador. Haz clic en la casilla de verificación situada a la derecha de **State(visible)**. Esto alterna el valor del estado **visible** entre **true** y **false** y actualiza el componente en el navegador en consecuencia.

Con esto concluimos nuestra exploración de la herramienta «**Components**». Pulsa **F5** para actualizar el navegador y que el componente «**Alert**» vuelva a aparecer antes de continuar. A continuación, exploraremos la herramienta «**Profiler**».

Uso de la herramienta Profiler

Ahora exploraremos el panel **Profiler**. Esta herramienta permite perfilar interacciones, lo cual es útil para rastrear problemas de rendimiento. Sigue estos pasos para perfilar el cierre de la alerta:

Antes de empezar a perfilar, nos aseguraremos de que la herramienta **Profiler** capture el motivo por el que se renderizan los componentes. Selecciona el panel **Profiler** y haz clic en la opción **Ver configuración** (el icono de la rueda dentada). Una vez que se abra la configuración de las herramientas de desarrollo de React, haz clic en la pestaña **Profiler** para ver la configuración **de Profiler**. Asegúrate de que la opción «**Registrar por qué se renderizó cada componente durante el perfilado**» esté marcada.

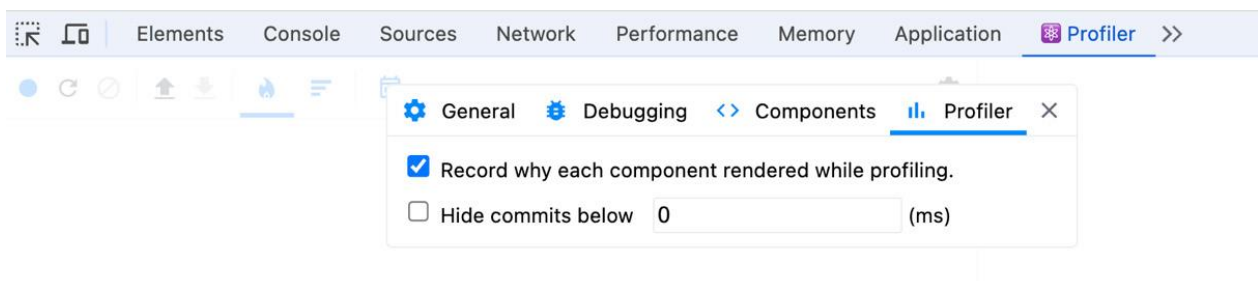


Figura 1.19 – Configuración de Profiler

Cierra la configuración y haz clic en la opción «**Iniciar perfilado**», que es el icono del círculo azul.

Haz clic en el botón *de cierre* de la alerta que aparece en la aplicación.

Haz clic en la opción «**Detener perfilado**», que es el icono con un círculo rojo.

Aparecerá una línea de tiempo con todas las actualizaciones de los componentes:

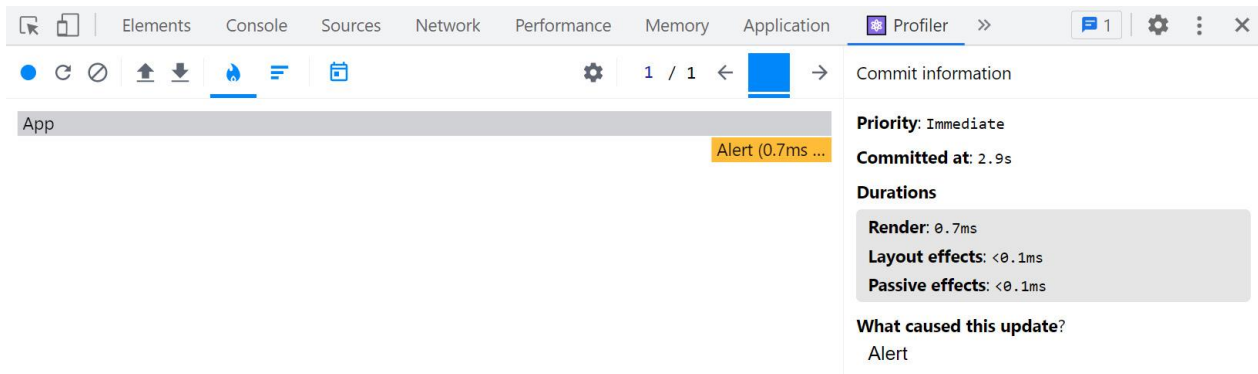


Figura 1.20 – Perfil del cierre de la alerta

Esto muestra que **la alerta** se volvió a renderizar al hacer clic en el botón *de cierre*, lo que tardó 0,7 milisegundos.

Esta herramienta resulta útil para detectar rápidamente los componentes lentos de una interacción del usuario.

Con esto concluimos nuestra exploración de las herramientas de desarrollo de React. A continuación, un resumen:

La herramienta de desarrollo React **Components** permite inspeccionar y probar las propiedades y el estado de los componentes

La herramienta de desarrollo React **Profiler** permite perfilar las interacciones de usuario con bajo rendimiento para ayudar a identificar el problema de raíz

Con esto llegamos al final del tema. A continuación, un resumen del tema.

Resumen

Ahora sabemos que React es una biblioteca popular para crear interfaces de usuario basadas en componentes. En este tema, hemos creado un componente **Alert** utilizando React.

La salida del componente se declara utilizando una mezcla de HTML y JavaScript llamada JSX. JSX debe transpilarse a JavaScript antes de que pueda ejecutarse en un navegador.

Las props se pueden pasar a un componente como atributos JSX. Esto permite a los usuarios del componente controlar su salida y su comportamiento. Un componente recibe las props como un parámetro de tipo objeto. Los nombres de los atributos JSX forman los nombres de las propiedades del objeto. En este tema hemos implementado una serie de props en el componente **Alert**.

Se pueden gestionar eventos para ejecutar lógica cuando el usuario interactúa con el componente. Hemos creado un controlador de eventos para el evento **de clic** del botón **de cierre** en el componente **Alert**.

El estado se puede utilizar para volver a renderizar un componente y actualizar su salida. El estado se define mediante el hook **useState** y, a menudo, se actualiza en los controladores de eventos. Hemos creado un estado para indicar si la alerta está visible.

Los eventos personalizados se pueden implementar como una propiedad de función. Esto permite a los usuarios del componente ejecutar lógica a medida que el usuario interactúa con él. Hemos implementado un evento de cierre en el componente **Alert**.

El componente **Alert** es un ejemplo de componente reutilizable que se puede utilizar en muchos lugares de una aplicación grande e incluso en diferentes aplicaciones.